Computer Olympic
POSN
KU Center

Topic : Heap

Tanaboon Tongbuasirilai
Department of Computer Science
Kasetsart University

Agenda

- Introduction
- Properties
- Applications
- Traversals
- Operations

Reference

- https://www.geeksforgeeks.org/heap-data-structure
- https://www.cs.yale.edu/homes/aspnes/pinewiki/Heaps.html
- https://mathworld.wolfram.com/Heap.html
- https://www.tutorialspoint.com/data_structures_algorithms/heap_data_structure.htm

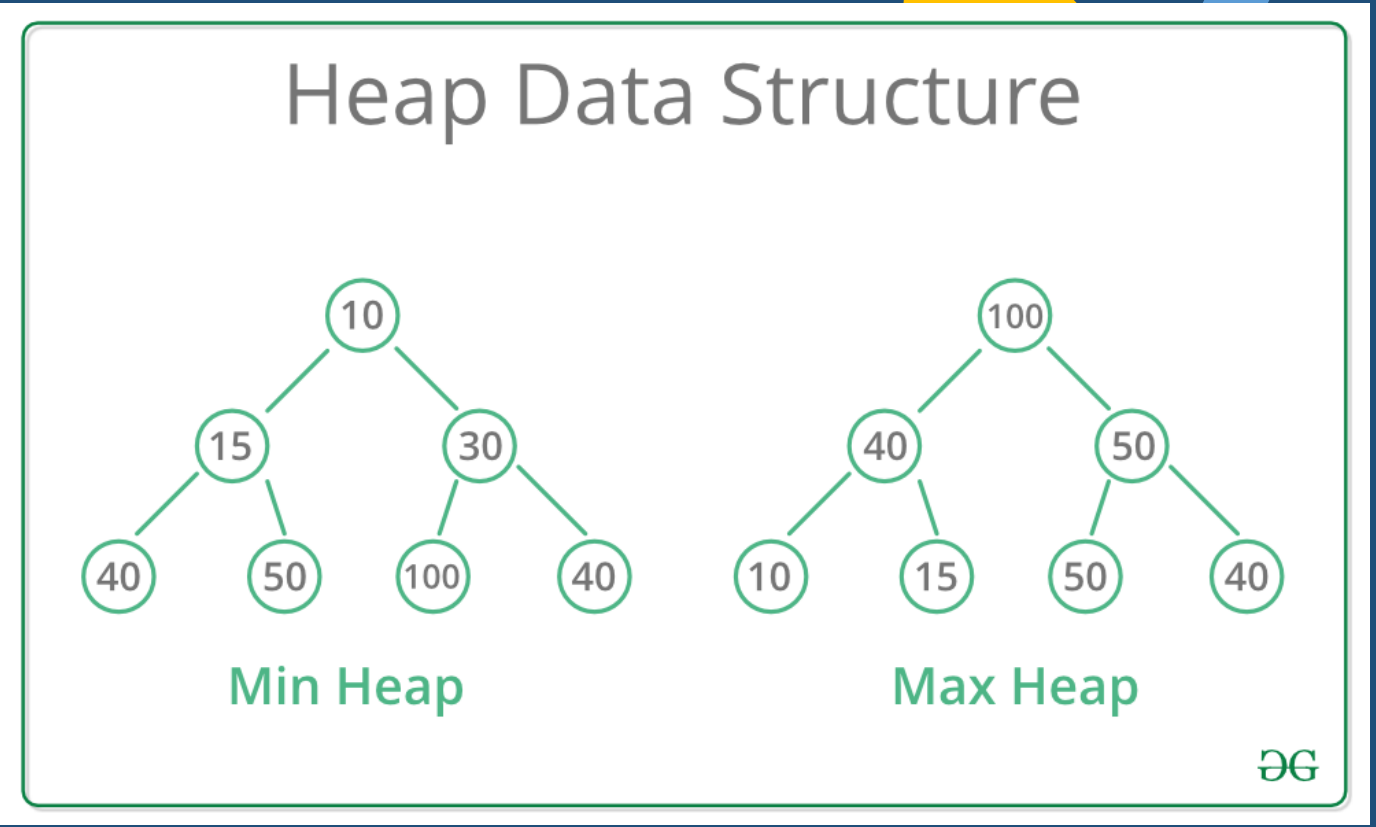
Introduction to Heap

- A heap is a binary tree in which each node has a smaller(or larger) key than its children; this property is called the heap property or heap invariant.
- Heaps are often used to implement priority queues, where the smallest (or largest) element is always at the root of the tree.
- A Heap is a special Tree-based Data Structure in which the tree is a complete binary tree.
- A heap can be viewed as a labeled binary tree in which the label of the ***i*** th node is smaller than the labels of any of its descendents.

Types of heaps

- Max-Heap:
  - In this heap, the value of the root node must be the greatest among all its child nodes and the same thing must be done for its left and right sub-tree also.

- Min-Heap:
  - In this heap, the value of the root node must be the smallest among all its child nodes and the same thing must be done for its left and right sub-tree also.



Heap properties

- **Complete Binary Tree:** A heap tree is a complete binary tree, meaning all levels of the tree are fully filled except possibly the last level, which is filled from left to right. This property ensures that the tree is efficiently represented using an array.

- **Heap Property:** This property ensures that the minimum (or maximum) element is always at the root of the tree according to the heap type.

- **Parent-Child Relationship:** The relationship between a parent node at index 'i' and its children is given by the formulas: left child at index 2i+1 and right child at index 2i+2 for 0-based indexing of node numbers.

- **Efficient Access to Extremal Elements:** The minimum or maximum element is always at the root of the heap, allowing constant-time access.

Operations

- Insertion
- Heapify
- Deletion

Coding – creating a heap

```c
// Structure to represent a heap
typedef struct {
    int* array;     // Array to store heap elements
    int capacity;   // Maximum capacity of the heap
    int size;       // Current size of the heap
} Heap;
```

1

```c
// Function to create a new heap
Heap* createHeap(int capacity)
{
    Heap* heap = (Heap*)malloc(sizeof(Heap));
    heap->array = (int*)malloc(capacity * sizeof(int));
    heap->capacity = capacity;
    heap->size = 0;
    return heap;
}
```

2

```c
// Example usage of the heap
int main()
{
    Heap* heap = createHeap(10);
    insert(heap, 35);
    insert(heap, 33);
    insert(heap, 42);
    insert(heap, 10);
    insert(heap, 14);
    insert(heap, 19);
    insert(heap, 27);
    insert(heap, 44);
    insert(heap, 26);
    insert(heap, 31);
```

3

Insertion

1. Increase the heap size by one.
2. Insert the new element k at the end of array A.
3. recursively swapping the new element k with its parent as necessary to restore the heap's property.

```c
// Function to insert a new element into the heap
void insert(Heap* heap, int value)
{
    if (heap->size == heap->capacity) {
        printf("Heap is full. Cannot insert more elements.\n");
        return;
    }
    // Insert the new element at the end
    int i = heap->size++;
    heap->array[i] = value;
    // Fix the heap property if it is violated
    while (i != 0 && heap->array[(i - 1) / 2] < heap->array[i]) {
        swap(&heap->array[i], &heap->array[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}
```

Max Heap construction

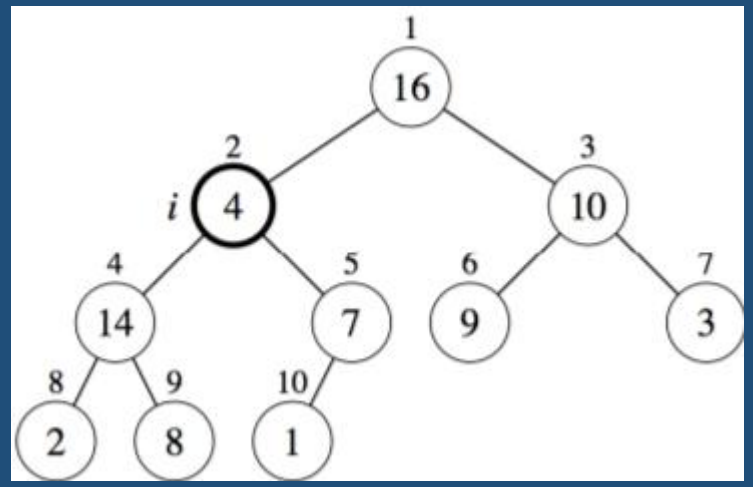
Input 35 33 42 10 14 19 27 44 26 31

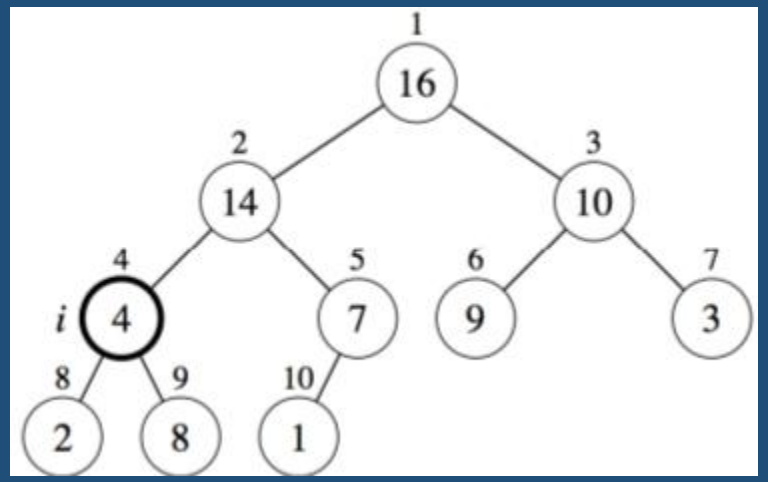
Heapify

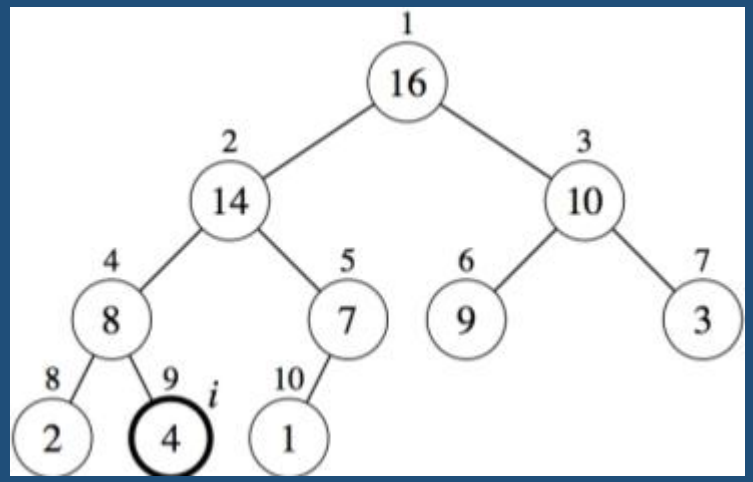
1. Compare left child with its root.
2. Compare right child with the largest.
3. Swap the largest if necessary.
4. Recursively heapify.

```
// Function to heapify a subtree rooted at index i
void heapify(Heap* heap, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    // Check if the left child is larger than the root
    if (left < heap->size && heap->array[left] > heap->array[largest])
        largest = left;
    // Check if the right child is larger than the largest so far
    if (right < heap->size && heap->array[right] > heap->array[largest])
        largest = right;
    // If the largest is not the root, swap the root with the largest
    if (largest != i) {
        swap(&heap->array[i], &heap->array[largest]);
        heapify(heap, largest);
    }
}
```

Example : Max - Heapify





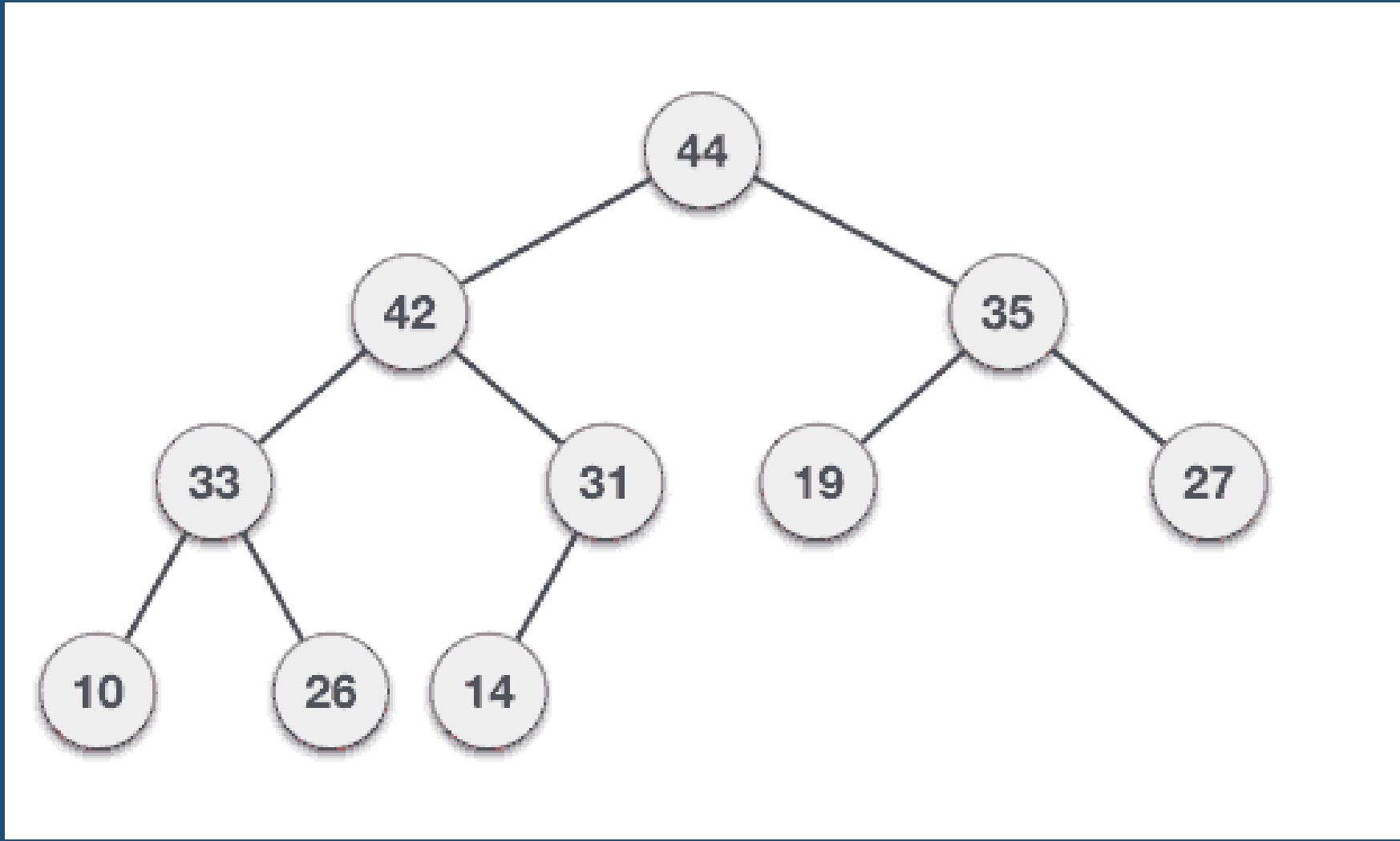


Deletion

1. Swap the root (maximum element) to be deleted with the last element in array A.
2. Delete the last element in array A.
3. Heapify the root.

```c
// delete the maximum element from the heap
int deleteMax(Heap* heap)
{
    if (heap->size == 0) {
        printf("Heap is empty. Cannot extract maximum element.\n");
        return -1;
    }
    // Store the root element
    int max = heap->array[0];
    // Replace the root with the last element
    heap->array[0] = heap->array[heap->size - 1];
    heap->size--;
    // Heapify the root
    heapify(heap, 0);
    return max;
}
```

Deletion



Complexity of operations

Operations	Complexity
insertion()	*O(log n)*
heapify()	*O(log n)*
deletion()	*O(log n)*

Problems on heap

(in grader1.cs.sci.ku.ac.th)

- [Heap] Stay Heap!